

Lecture 03 - Geometric Transformations

This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Source chunk	course_text_parts/03_lectures/03_geometric-transformations.txt
Extracted slide pages	54
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Untitled slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Untitled slide' introduces a concrete graphics object, operation, or relation. The named terms describe input data, a processing step, and an output used elsewhere in the rendering workflow. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. The object-level relation is therefore input, operation, output, and the later graphics stage that consumes the output.
Technical commentary	This slide is about Untitled slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Untitled slide' into a causal sentence instead of repeating the slide title?
Page 2	Transforming 3D Models
Source cue	- 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through

Professor-style explanation	The slide 'Transforming 3D Models' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Transforming 3D Models. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - 3D models need to be positioned and oriented in the virtual world / (sometimes scaling and other modifications are necessary) / - These modifications are performed by applying transformations to the / vertices describing a 3D model / - Transformations are applied through
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Transforming 3D Models'?

Page 3	3.1 Mathematical Foundations
Source cue	3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL
Professor-style explanation	The slide '3.1 Mathematical Foundations' describes the OpenGL side of the rendering pipeline. OpenGL represents rendering through objects such as contexts, buffers, vertex arrays, shaders, textures, samplers, and framebuffers. At draw time, the currently bound objects and state determine what data reaches the GPU and how the pipeline processes it. On the slide, the concrete items are: 3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL. The concrete relation is between API state, GPU resource, shader input, and final rendering result.
Technical commentary	This slide is about 3.1 Mathematical Foundations. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: 3.2 Affine Transformations / 3.3 Composition of Transformations / 3.4 Coordinate System Change / 3.5 Transformations in OpenGL
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '3.1 Mathematical Foundations'?

Page 4	3.1 Mathematical Foundations
Source cue	Vector and Matrix Calculus

Professor-style explanation	The slide '3.1 Mathematical Foundations' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: Vector and Matrix Calculus. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about 3.1 Mathematical Foundations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Vector and Matrix Calculus
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3.1 Mathematical Foundations'?

Page 5	Vertices as Vectors
Source cue	- Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of $\mathbb{R}^2, \mathbb{R}^3, \mathbb{R}^4$ / - In computer graphics mostly $\mathbb{R}^3$ or $\mathbb{R}^4$ vectors used / - Vertices of a polygon are represented as column vectors / $x \in \mathbb{R}^n$
Professor-style explanation	The slide 'Vertices as Vectors' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of $\mathbb{R}^2, \mathbb{R}^3, \mathbb{R}^4$ / - In computer graphics mostly $\mathbb{R}^3$ or $\mathbb{R}^4$ vectors used / - Vertices of a polygon are represented as column vectors / $x \in \mathbb{R}^n$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Vertices as Vectors. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Vectors are used to represent geometric objects (coordinates) / - Vectors have real numbers as components, so they are elements of $\mathbb{R}^2, \mathbb{R}^3, \mathbb{R}^4$ / - In computer graphics mostly $\mathbb{R}^3$ or $\mathbb{R}^4$ vectors used / - Vertices of a polygon are represented as column vectors / $x \in \mathbb{R}^n$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Vertices as Vectors'?

Page 6	Transformations as Matrices
Source cue	- Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} / \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} / \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} =$

Professor-style explanation	The slide 'Transformations as Matrices' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $\blacksquare \blacksquare \blacksquare \blacksquare / \blacksquare \blacksquare \blacksquare \blacksquare / \blacksquare =$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Transformations as Matrices. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Matrices are used to represent transformations / - In computer graphics mainly 4×4 square matrices are used / $\blacksquare \blacksquare \blacksquare \blacksquare / \blacksquare \blacksquare \blacksquare \blacksquare / \blacksquare =$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Transformations as Matrices'?

Page 7	Matrix Vector Multiplication
Source cue	- Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $\blacksquare \times \blacksquare$ matrix $\blacksquare$ and $\blacksquare m$ vector $\blacksquare$ / - Result: $\blacksquare n$ vector $\blacksquare$, whereby is $\blacksquare$ transformed through $\blacksquare$ / - Example for 4×4 matrix $\blacksquare$ and $\blacksquare 4$ vector $\blacksquare$:
Professor-style explanation	The slide 'Matrix Vector Multiplication' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $\blacksquare \times \blacksquare$ matrix $\blacksquare$ and $\blacksquare m$ vector $\blacksquare$ / - Result: $\blacksquare n$ vector $\blacksquare$, whereby is $\blacksquare$ transformed through $\blacksquare$ / - Example for 4×4 matrix $\blacksquare$ and $\blacksquare 4$ vector $\blacksquare$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Matrix Vector Multiplication. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Since vectors describe vertices and matrices describe transformations, / transformations can be applied through matrix vector multiplication / - Input: $\blacksquare \times \blacksquare$ matrix $\blacksquare$ and $\blacksquare m$ vector $\blacksquare$ / - Result: $\blacksquare n$ vector $\blacksquare$, whereby is $\blacksquare$ transformed through $\blacksquare$ / - Example for 4×4 matrix $\blacksquare$ and $\blacksquare 4$ vector $\blacksquare$:
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Matrix Vector Multiplication'?

Page 8	3.2 Affine Transformations
--------	----------------------------

Source cue	Translation, Rotation, Scaling, and more
Professor-style explanation	The slide '3.2 Affine Transformations' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: Translation, Rotation, Scaling, and more. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about 3.2 Affine Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Translation, Rotation, Scaling, and more
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3.2 Affine Transformations'?
Page 9	Translations – Moving 3D Models
Source cue	- Matrix representation of a translation / $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$
Professor-style explanation	The slide 'Translations – Moving 3D Models' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Matrix representation of a translation / $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Translations – Moving 3D Models. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Matrix representation of a translation / $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Translations – Moving 3D Models'?
Page 10	Translation Properties
Source cue	- Lengths and angles are preserved / - Identity: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ (identity matrix) / - Changing order of translations: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix}$

Professor-style explanation	The slide 'Translation Properties' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Lengths and angles are preserved / - Identity: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ (identity matrix) / - Changing order of translations: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ / $x y z x y z x y z x y z$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Translation Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Lengths and angles are preserved / - Identity: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ (identity matrix) / - Changing order of translations: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ / $x y z x y z x y z x y z$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Translation Properties'?
Page 11	Scaling – Resizing 3D Models
Source cue	- Matrix representation of a scaling / $\begin{pmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & s \end{pmatrix}$, s , s = / $x y z$
Professor-style explanation	The slide 'Scaling – Resizing 3D Models' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Matrix representation of a scaling / $\begin{pmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & s \end{pmatrix}$, s , s = / $x y z$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Scaling – Resizing 3D Models. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Matrix representation of a scaling / $\begin{pmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & s \end{pmatrix}$, s , s = / $x y z$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Scaling – Resizing 3D Models'?
Page 12	Scaling Properties
Source cue	- Lengths are not preserved, angles only for uniform scalings / - Identity: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ (identity matrix) / - Changing order of scalings: $\begin{pmatrix} s_1 & 0 & 0 \\ 0 & s_1 & 0 \\ 0 & 0 & s_1 \end{pmatrix} \cdot \begin{pmatrix} s_2 & 0 & 0 \\ 0 & s_2 & 0 \\ 0 & 0 & s_2 \end{pmatrix} = \begin{pmatrix} s_2 & 0 & 0 \\ 0 & s_2 & 0 \\ 0 & 0 & s_2 \end{pmatrix} \cdot \begin{pmatrix} s_1 & 0 & 0 \\ 0 & s_1 & 0 \\ 0 & 0 & s_1 \end{pmatrix}$ / $x y z x y z x y z x y z$

Professor-style explanation	The slide 'Scaling Properties' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Lengths are not preserved, angles only for uniform scalings / - Identity: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ (identity matrix) / - Changing order of scalings: $\begin{pmatrix} s_1 & 0 & 0 \\ 0 & s_1 & 0 \\ 0 & 0 & s_1 \end{pmatrix} \cdot \begin{pmatrix} s_2 & 0 & 0 \\ 0 & s_2 & 0 \\ 0 & 0 & s_2 \end{pmatrix} = \begin{pmatrix} s_2 & 0 & 0 \\ 0 & s_2 & 0 \\ 0 & 0 & s_2 \end{pmatrix} \cdot \begin{pmatrix} s_1 & 0 & 0 \\ 0 & s_1 & 0 \\ 0 & 0 & s_1 \end{pmatrix}$ / x y z x y z x y z x y z. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Scaling Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Lengths are not preserved, angles only for uniform scalings / - Identity: $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ (identity matrix) / - Changing order of scalings: $\begin{pmatrix} s_1 & 0 & 0 \\ 0 & s_1 & 0 \\ 0 & 0 & s_1 \end{pmatrix} \cdot \begin{pmatrix} s_2 & 0 & 0 \\ 0 & s_2 & 0 \\ 0 & 0 & s_2 \end{pmatrix} = \begin{pmatrix} s_2 & 0 & 0 \\ 0 & s_2 & 0 \\ 0 & 0 & s_2 \end{pmatrix} \cdot \begin{pmatrix} s_1 & 0 & 0 \\ 0 & s_1 & 0 \\ 0 & 0 & s_1 \end{pmatrix}$ / x y z x y z x y z x y z
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Scaling Properties'?
Page 13	Rotation – Changing Orientation of 3D Models 1/2
Source cue	- Matrix representation of a rotation around x-, y- and z-axis / $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ / $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ = / 0 $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ 0
Professor-style explanation	The slide 'Rotation – Changing Orientation of 3D Models 1/2' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Matrix representation of a rotation around x-, y- and z-axis / $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ / $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ = / 0 $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ 0. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Rotation – Changing Orientation of 3D Models 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Matrix representation of a rotation around x-, y- and z-axis / $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ / $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ = / 0 $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ 0
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Rotation – Changing Orientation of 3D Models 1/2'?
Page 14	Rotation – Changing Orientation of 3D Models 2/2
Source cue	- Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x

Professor-style explanation	The slide 'Rotation – Changing Orientation of 3D Models 2/2' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Rotation – Changing Orientation of 3D Models 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Rotations are made in the right-handed coordinate system so that when we / look along a positive axis to the origin, a 90° rotation about that axis / interleaves the positive axes as follows / - Rotation around x-axis transforms +y to +z / - Rotation around y-axis transforms +z to +x
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Rotation – Changing Orientation of 3D Models 2/2'?

Page 15	Rotation Properties
Source cue	- Lengths and angles are preserved / - Identity: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ / - Changing order of rotations around same axis: / - $\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos^2 \theta & -2\sin \theta \cos \theta \\ 0 & 2\sin \theta \cos \theta & \cos^2 \theta \end{bmatrix}$
Professor-style explanation	The slide 'Rotation Properties' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Lengths and angles are preserved / - Identity: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ / - Changing order of rotations around same axis: / - $\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos^2 \theta & -2\sin \theta \cos \theta \\ 0 & 2\sin \theta \cos \theta & \cos^2 \theta \end{bmatrix}$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Rotation Properties. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Lengths and angles are preserved / - Identity: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ / - Changing order of rotations around same axis: / - $\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos^2 \theta & -2\sin \theta \cos \theta \\ 0 & 2\sin \theta \cos \theta & \cos^2 \theta \end{bmatrix}$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Rotation Properties'?

Page 16	Mirroring – Reflecting 3D Models on a Plane
---------	---

Source cue	- Matrix representation of mirroring along x-y plane / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix} =$
Professor-style explanation	The slide 'Mirroring – Reflecting 3D Models on a Plane' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Matrix representation of mirroring along x-y plane / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix} =$ / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix}$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Mirroring – Reflecting 3D Models on a Plane. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Matrix representation of mirroring along x-y plane / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix} =$ / $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix}$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Mirroring – Reflecting 3D Models on a Plane'?
Page 17	Shearing – Skewing 3D Models
Source cue	- Matrix representation of a shearing / $\begin{bmatrix} 1 & \blacksquare & \blacksquare & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} / xy\ xz / \begin{bmatrix} 1 & \blacksquare & \blacksquare & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} / yx\ yz$
Professor-style explanation	The slide 'Shearing – Skewing 3D Models' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Matrix representation of a shearing / $\begin{bmatrix} 1 & \blacksquare & \blacksquare & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} / xy\ xz / \begin{bmatrix} 1 & \blacksquare & \blacksquare & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} / yx\ yz$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Shearing – Skewing 3D Models. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Matrix representation of a shearing / $\begin{bmatrix} 1 & \blacksquare & \blacksquare & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} / xy\ xz / \begin{bmatrix} 1 & \blacksquare & \blacksquare & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} / yx\ yz$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Shearing – Skewing 3D Models'?
Page 18	3.3 Composition of Transformations
Source cue	Concatenation of several transformation steps

Professor-style explanation	The slide '3.3 Composition of Transformations' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: Concatenation of several transformation steps. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about 3.3 Composition of Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Concatenation of several transformation steps
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3.3 Composition of Transformations'?

Page 19	Composition of Transformations
Source cue	- Composition of transformations $\mathbf{T}$ is done / by multiplying the respective transformation matrices / - $(\mathbf{T}_1 \cdot \mathbf{T}_2 \cdot \dots \cdot \mathbf{T}_{n-1} \cdot \mathbf{T}_n) \cdot \mathbf{P} = (\mathbf{T}_1 \cdot (\mathbf{T}_2 \cdot (\dots (\mathbf{T}_{n-1} \cdot (\mathbf{T}_n \cdot \mathbf{P}))))$ / $\mathbf{T}_1 \mathbf{T}_2 \dots \mathbf{T}_{n-1} \mathbf{T}_n$ / - Interpretation: apply first transformation $\mathbf{T}_1$, then $\mathbf{T}_2$, ..., and finally $\mathbf{T}_n$
Professor-style explanation	The slide 'Composition of Transformations' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Composition of transformations $\mathbf{T}$ is done / by multiplying the respective transformation matrices / - $(\mathbf{T}_1 \cdot \mathbf{T}_2 \cdot \dots \cdot \mathbf{T}_{n-1} \cdot \mathbf{T}_n) \cdot \mathbf{P} = (\mathbf{T}_1 \cdot (\mathbf{T}_2 \cdot (\dots (\mathbf{T}_{n-1} \cdot (\mathbf{T}_n \cdot \mathbf{P}))))$ / $\mathbf{T}_1 \mathbf{T}_2 \dots \mathbf{T}_{n-1} \mathbf{T}_n$ / - Interpretation: apply first transformation $\mathbf{T}_1$, then $\mathbf{T}_2$, ..., and finally $\mathbf{T}_n$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Composition of Transformations. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Composition of transformations $\mathbf{T}$ is done / by multiplying the respective transformation matrices / - $(\mathbf{T}_1 \cdot \mathbf{T}_2 \cdot \dots \cdot \mathbf{T}_{n-1} \cdot \mathbf{T}_n) \cdot \mathbf{P} = (\mathbf{T}_1 \cdot (\mathbf{T}_2 \cdot (\dots (\mathbf{T}_{n-1} \cdot (\mathbf{T}_n \cdot \mathbf{P}))))$ / $\mathbf{T}_1 \mathbf{T}_2 \dots \mathbf{T}_{n-1} \mathbf{T}_n$ / - Interpretation: apply first transformation $\mathbf{T}_1$, then $\mathbf{T}_2$, ..., and finally $\mathbf{T}_n$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Composition of Transformations'?
Page 20	Reference Points 1/2
Source cue	- Rotation and scaling happen in relation to origin (reference point)

Professor-style explanation	The slide 'Reference Points 1/2' introduces a concrete graphics object, operation, or relation. The named terms describe input data, a processing step, and an output used elsewhere in the rendering workflow. On the slide, the concrete items are: - Rotation and scaling happen in relation to origin (reference point). The object-level relation is therefore input, operation, output, and the later graphics stage that consumes the output.
Technical commentary	This slide is about Reference Points 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Rotation and scaling happen in relation to origin (reference point)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Reference Points 1/2' into a causal sentence instead of repeating the slide title?

Page 21	Reference Points 2/2
Source cue	- In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point ■ to the origin ■ / - Rotate (or scale) the 3D model / - Move the 3D model back to ■
Professor-style explanation	The slide 'Reference Points 2/2' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point ■ to the origin ■ / - Rotate (or scale) the 3D model / - Move the 3D model back to ■. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Reference Points 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - In order to rotate (or scale) 3D models at an arbitrary reference point, / a sequence of transformations is necessary / - Move the 3D model from reference point ■ to the origin ■ / - Rotate (or scale) the 3D model / - Move the 3D model back to ■
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Reference Points 2/2'?

Page 22	Rotation Around Arbitrary Axis
Source cue	- Rotation around any axis / - Axis given by center of rotation ■ and direction ■ = (■ , ■ , ■) / x y z / - Rotation by ■ degrees / - Construction of transformation as transformation sequence

Professor-style explanation	The slide 'Rotation Around Arbitrary Axis' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Rotation around any axis / - Axis given by center of rotation $\mathbf{c}$ and direction $\mathbf{u} = (u_x, u_y, u_z) / x y z$ / - Rotation by θ degrees / - Construction of transformation as transformation sequence. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Rotation Around Arbitrary Axis. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Rotation around any axis / - Axis given by center of rotation $\mathbf{c}$ and direction $\mathbf{u} = (u_x, u_y, u_z) / x y z$ / - Rotation by θ degrees / - Construction of transformation as transformation sequence
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Rotation Around Arbitrary Axis'?
Page 23	Reminder: Vector Calculus
Source cue	- Length of a vector is expressed as follows / $ \mathbf{v} = \sqrt{v_x^2 + v_y^2 + v_z^2} / x y z$ / - Vectors of length $ \mathbf{v} = 1$ are normalized and called unit vectors / - Arbitrary vectors can be normalized as follows
Professor-style explanation	The slide 'Reminder: Vector Calculus' explains local illumination at a surface point. The surface normal defines orientation, the light vector defines incoming light, the view vector defines the observer, and material parameters scale ambient, diffuse, or specular terms. On the slide, the concrete items are: - Length of a vector is expressed as follows / $ \mathbf{v} = \sqrt{v_x^2 + v_y^2 + v_z^2} / x y z$ / - Vectors of length $ \mathbf{v} = 1$ are normalized and called unit vectors / - Arbitrary vectors can be normalized as follows. The object-level relation is light source, surface point, material response, and computed color.
Technical commentary	This slide is about Reminder: Vector Calculus. The lighting objects on the slide combine normals, light directions, view directions, material coefficients, and shading locations to compute color. Concrete items shown: - Length of a vector is expressed as follows / $ \mathbf{v} = \sqrt{v_x^2 + v_y^2 + v_z^2} / x y z$ / - Vectors of length $ \mathbf{v} = 1$ are normalized and called unit vectors / - Arbitrary vectors can be normalized as follows
Why it matters	Lighting formulas are only meaningful when their vectors and material terms are interpreted correctly.
Check yourself	Can you identify the normal, light vector, view vector, and material term relevant to 'Reminder: Vector Calculus'?
Page 24	Reminder: Cross Product (= vector product)
Source cue	- Given two vectors $\mathbf{u}$ and $\mathbf{v}$ in $\mathbb{R}^3$ / - Cross product $\mathbf{u} \times \mathbf{v}$ of $\mathbf{u}$ and $\mathbf{v}$ is a vector defined as follows / $\mathbf{u} \times \mathbf{v} = \begin{vmatrix} \mathbf{e}_x & \mathbf{e}_y & \mathbf{e}_z \\ u_x & u_y & u_z \\ v_x & v_y & v_z \end{vmatrix} = \begin{vmatrix} u_y v_z - u_z v_y \\ u_z v_x - u_x v_z \\ u_x v_y - u_y v_x \end{vmatrix} / x y z$ / $\mathbf{u} \times \mathbf{v} = \mathbf{u} \cdot \mathbf{v} - \mathbf{v} \cdot \mathbf{u}$

Professor-style explanation	The slide 'Reminder: Cross Product (= vector product)' introduces a concrete graphics object, operation, or relation. The named terms describe input data, a processing step, and an output used elsewhere in the rendering workflow. On the slide, the concrete items are: - Given two vectors $\mathbf{u}$ and $\mathbf{v}$ in $\mathbb{R}^3$ / - Cross product $\mathbf{u} \times \mathbf{v}$ of $\mathbf{u}$ and $\mathbf{v}$ is a vector defined as follows / $\mathbf{u} \times \mathbf{v} = \begin{pmatrix} u_y v_z - u_z v_y \\ u_z v_x - u_x v_z \\ u_x v_y - u_y v_x \end{pmatrix}$ / $\mathbf{u} \times \mathbf{v} = \mathbf{u} \times \mathbf{v}$ / $\mathbf{u} \times \mathbf{v} = \mathbf{u} \times \mathbf{v}$. The object-level relation is therefore input, operation, output, and the later graphics stage that consumes the output.
Technical commentary	This slide is about Reminder: Cross Product (= vector product). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Given two vectors $\mathbf{u}$ and $\mathbf{v}$ in $\mathbb{R}^3$ / - Cross product $\mathbf{u} \times \mathbf{v}$ of $\mathbf{u}$ and $\mathbf{v}$ is a vector defined as follows / $\mathbf{u} \times \mathbf{v} = \begin{pmatrix} u_y v_z - u_z v_y \\ u_z v_x - u_x v_z \\ u_x v_y - u_y v_x \end{pmatrix}$ / $\mathbf{u} \times \mathbf{v} = \mathbf{u} \times \mathbf{v}$ / $\mathbf{u} \times \mathbf{v} = \mathbf{u} \times \mathbf{v}$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Reminder: Cross Product (= vector product)' into a causal sentence instead of repeating the slide title?

Page 25	Example – Transforming Directed Line Segment 1/7
Source cue	- Example transformation task / - Given line segments $\mathbf{u}$ and $\mathbf{v}$ / 1 2 1 3 / - Transform it into the yz-plane so that $\mathbf{u}$ lies on the z-axis / 1 2
Professor-style explanation	The slide 'Example – Transforming Directed Line Segment 1/7' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Example transformation task / - Given line segments $\mathbf{u}$ and $\mathbf{v}$ / 1 2 1 3 / - Transform it into the yz-plane so that $\mathbf{u}$ lies on the z-axis / 1 2. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example – Transforming Directed Line Segment 1/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Example transformation task / - Given line segments $\mathbf{u}$ and $\mathbf{v}$ / 1 2 1 3 / - Transform it into the yz-plane so that $\mathbf{u}$ lies on the z-axis / 1 2
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example – Transforming Directed Line Segment 1/7'?

Page 26	Example – Transforming Directed Line Segment 2/7
Source cue	- Approach 1: Determine rotation angles for rotation sequence / - Move $\mathbf{u} = x, y, z$ into the origin / 1 1 1 1 / - Rotate around the y-axis so that $\mathbf{u}$ lies in the yz-plane / 1 2

Professor-style explanation	The slide 'Example – Transforming Directed Line Segment 2/7' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Approach 1: Determine rotation angles for rotation sequence / - Move $\mathbf{p} = x, y, z$ into the origin / $1 \ 1 \ 1 \ 1$ / - Rotate around the y-axis so that $\mathbf{p}$ lies in the yz-plane / $1 \ 2$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example – Transforming Directed Line Segment 2/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Approach 1: Determine rotation angles for rotation sequence / - Move $\mathbf{p} = x, y, z$ into the origin / $1 \ 1 \ 1 \ 1$ / - Rotate around the y-axis so that $\mathbf{p}$ lies in the yz-plane / $1 \ 2$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example – Transforming Directed Line Segment 2/7'?
Page 27	Example – Transforming Directed Line Segment 3/7
Source cue	- Step 2 – Rotation of $\mathbf{p}'$ around y-axis into yz-plane / $1 \ 2$ / - Rotation $\mathbf{p}'$ ($\mathbf{p}' - 90^\circ$) / - Rotation angle – ($90^\circ - \mathbf{p}'$) = $\mathbf{p}' - 90^\circ$ / - Matrix ingredients
Professor-style explanation	The slide 'Example – Transforming Directed Line Segment 3/7' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Step 2 – Rotation of $\mathbf{p}'$ around y-axis into yz-plane / $1 \ 2$ / - Rotation $\mathbf{p}'$ ($\mathbf{p}' - 90^\circ$) / - Rotation angle – ($90^\circ - \mathbf{p}'$) = $\mathbf{p}' - 90^\circ$ / - Matrix ingredients. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example – Transforming Directed Line Segment 3/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Step 2 – Rotation of $\mathbf{p}'$ around y-axis into yz-plane / $1 \ 2$ / - Rotation $\mathbf{p}'$ ($\mathbf{p}' - 90^\circ$) / - Rotation angle – ($90^\circ - \mathbf{p}'$) = $\mathbf{p}' - 90^\circ$ / - Matrix ingredients
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example – Transforming Directed Line Segment 3/7'?
Page 28	Example – Transforming Directed Line Segment 4/7
Source cue	- Step 3 – Rotation of $\mathbf{p}''$ around x-axis onto z-axis / $1 \ 2$ / - Rotation $\mathbf{p}''$ (ϕ) / - Matrix ingredients / z''

Professor-style explanation	The slide 'Example – Transforming Directed Line Segment 4/7' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Step 3 – Rotation of $\mathbf{p}''$ $\mathbf{v}''$ around x-axis onto z-axis / 1 2 / - Rotation $\mathbf{R}(\phi)$ / - Matrix ingredients / $\mathbf{z}''$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example – Transforming Directed Line Segment 4/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Step 3 – Rotation of $\mathbf{p}''$ $\mathbf{v}''$ around x-axis onto z-axis / 1 2 / - Rotation $\mathbf{R}(\phi)$ / - Matrix ingredients / $\mathbf{z}''$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example – Transforming Directed Line Segment 4/7'?

Page 29	Example – Transforming Directed Line Segment 5/7
Source cue	- Step 4 – Rotation of $\mathbf{p}'''$ $\mathbf{v}'''$ into yz-plane / 1 3 / - Rotation $\mathbf{R}(\alpha)$ / $\mathbf{y}'''$ $\mathbf{z}'''$ / - Matrix ingredients
Professor-style explanation	The slide 'Example – Transforming Directed Line Segment 5/7' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Step 4 – Rotation of $\mathbf{p}'''$ $\mathbf{v}'''$ into yz-plane / 1 3 / - Rotation $\mathbf{R}(\alpha)$ / $\mathbf{y}'''$ $\mathbf{z}'''$ / - Matrix ingredients. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example – Transforming Directed Line Segment 5/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Step 4 – Rotation of $\mathbf{p}'''$ $\mathbf{v}'''$ into yz-plane / 1 3 / - Rotation $\mathbf{R}(\alpha)$ / $\mathbf{y}'''$ $\mathbf{z}'''$ / - Matrix ingredients
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example – Transforming Directed Line Segment 5/7'?

Page 30	Example – Transforming Directed Line Segment 6/7
Source cue	- Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / ($\mathbf{R}_x$ on x-axis, $\mathbf{R}_y$ on y-axis, $\mathbf{R}_z$ on z-axis)

Professor-style explanation	The slide 'Example – Transforming Directed Line Segment 6/7' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / (■ on x-axis, ■ on y-axis, ■ on z-axis). The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example – Transforming Directed Line Segment 6/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Approach 2: Construct orthogonal matrix which obeys following properties / - Each row and column corresponds to a unit length vector / - Row and column vectors are orthogonal to each other / - Each row vector rotates on one main axis / (■ on x-axis, ■ on y-axis, ■ on z-axis)
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example – Transforming Directed Line Segment 6/7'?

Page 31	Example – Transforming Directed Line Segment 7/7
Source cue	- Consideration 2: Formulate ■ as unit vector / orthogonal to the plane spanned by ■ , ■ , and ■ / $1 \ 2 \ 3 / \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \end{bmatrix} \times \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \end{bmatrix} / - \blacksquare = \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \end{bmatrix} = 1 \ 3 \ 1 \ 2$
Professor-style explanation	The slide 'Example – Transforming Directed Line Segment 7/7' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Consideration 2: Formulate ■ as unit vector / orthogonal to the plane spanned by ■ , ■ , and ■ / $1 \ 2 \ 3 / \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \end{bmatrix} \times \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \end{bmatrix} / - \blacksquare = \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \end{bmatrix} = 1 \ 3 \ 1 \ 2$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example – Transforming Directed Line Segment 7/7. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Consideration 2: Formulate ■ as unit vector / orthogonal to the plane spanned by ■ , ■ , and ■ / $1 \ 2 \ 3 / \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \end{bmatrix} \times \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \end{bmatrix} / - \blacksquare = \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \end{bmatrix} = 1 \ 3 \ 1 \ 2$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example – Transforming Directed Line Segment 7/7'?

Page 32	Example – Direction of Flight Transformation 1/3
---------	--

Professor-style explanation	The slide 'Example – Direction of Flight Transformation 3/3' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Transformation $\mathbf{T}$ is described by a translation and an orthogonal matrix $\mathbf{R} = \begin{bmatrix} 0 & 1x & 2x & 3x \\ 0 & 1y & 2y & 3y \end{bmatrix}$. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example – Direction of Flight Transformation 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Transformation $\mathbf{T}$ is described by a translation and an orthogonal matrix $\mathbf{R} = \begin{bmatrix} 0 & 1x & 2x & 3x \\ 0 & 1y & 2y & 3y \end{bmatrix}$
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example – Direction of Flight Transformation 3/3'?

Page 35	3.4 Coordinate System Change
Source cue	Transformation as change of frame of reference
Professor-style explanation	The slide '3.4 Coordinate System Change' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: Transformation as change of frame of reference. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about 3.4 Coordinate System Change. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: Transformation as change of frame of reference
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '3.4 Coordinate System Change'?

Page 36	Coordinate System Change
Source cue	- Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2

Professor-style explanation	The slide 'Coordinate System Change' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Coordinate System Change. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Alternative view on transformations: / - transformations change the coordinate system of 3D model / Example 1 Example 2
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Coordinate System Change'?

Page 37	Local Coordinate Systems
Source cue	- Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS)
Professor-style explanation	The slide 'Local Coordinate Systems' describes the OpenGL side of the rendering pipeline. OpenGL represents rendering through objects such as contexts, buffers, vertex arrays, shaders, textures, samplers, and framebuffers. At draw time, the currently bound objects and state determine what data reaches the GPU and how the pipeline processes it. On the slide, the concrete items are: - Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS). The concrete relation is between API state, GPU resource, shader input, and final rendering result.
Technical commentary	This slide is about Local Coordinate Systems. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Application in OpenGL / - Individual 3D models of a scene have their own, local coordinate system / - Local coordinate systems of the 3D models must be changed / into a common coordinate system, the world coordinate system / - Transformation between coordinate systems (CS)
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Local Coordinate Systems'?

Page 38	Example 1 – Coordinate System Change 1/2
---------	--

Source cue	$x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center ■ / $y(1)$ / and subsequent translation to a point ■
Professor-style explanation	The slide 'Example 1 – Coordinate System Change 1/2' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: $x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center ■ / $y(1)$ / and subsequent translation to a point ■. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example 1 – Coordinate System Change 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center ■ / $y(1)$ / and subsequent translation to a point ■
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 1 – Coordinate System Change 1/2'?
Page 39	Example 1 – Coordinate System Change 2/2
Source cue	$y(5)$ / $y(2)$ $y(3)$ / $x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center ■ / $y(1)$ / and subsequent translation to a point ■
Professor-style explanation	The slide 'Example 1 – Coordinate System Change 2/2' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: $y(5)$ / $y(2)$ $y(3)$ / $x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center ■ / $y(1)$ / and subsequent translation to a point ■. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example 1 – Coordinate System Change 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: $y(5)$ / $y(2)$ $y(3)$ / $x(1)$ / - Example transformation task $y(5)$ / - Rotation and scaling of an object around its center ■ / $y(1)$ / and subsequent translation to a point ■
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 1 – Coordinate System Change 2/2'?
Page 40	Example 2 – Hierarchical Coordinate Systems 1/2
Source cue	- To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system

Professor-style explanation	The slide 'Example 2 – Hierarchical Coordinate Systems 1/2' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example 2 – Hierarchical Coordinate Systems 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - To model complex scenes, a hierarchy of coordinate systems is helpful / - Example: modeling of a tricycle / - World-coordinate system / - Tricycle-coordinate system / - Wheel-coordinate system
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 2 – Hierarchical Coordinate Systems 1/2'?
Page 41	Example 2 – Hierarchical Coordinate Systems 1/2
Source cue	- Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation R (α) / y-tr
Professor-style explanation	The slide 'Example 2 – Hierarchical Coordinate Systems 1/2' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation R (α) / y-tr. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Example 2 – Hierarchical Coordinate Systems 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Scene graphs are used to represent hierarchical coordinate systems / and the required transformations / Tricycle Translation T / Steering Rotation R (α) / y-tr
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Example 2 – Hierarchical Coordinate Systems 1/2'?
Page 42	3.5 Transformations in OpenGL
Source cue	Implementation with GLM und GLSL

Professor-style explanation	The slide '3.5 Transformations in OpenGL' describes the OpenGL side of the rendering pipeline. OpenGL represents rendering through objects such as contexts, buffers, vertex arrays, shaders, textures, samplers, and framebuffers. At draw time, the currently bound objects and state determine what data reaches the GPU and how the pipeline processes it. On the slide, the concrete items are: Implementation with GLM und GLSL. The concrete relation is between API state, GPU resource, shader input, and final rendering result.
Technical commentary	This slide is about 3.5 Transformations in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: Implementation with GLM und GLSL
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '3.5 Transformations in OpenGL'?

Page 43	Geometric Transformations in OpenGL
Source cue	- Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements
Professor-style explanation	The slide 'Geometric Transformations in OpenGL' describes the OpenGL side of the rendering pipeline. OpenGL represents rendering through objects such as contexts, buffers, vertex arrays, shaders, textures, samplers, and framebuffers. At draw time, the currently bound objects and state determine what data reaches the GPU and how the pipeline processes it. On the slide, the concrete items are: - Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements. The concrete relation is between API state, GPU resource, shader input, and final rendering result.
Technical commentary	This slide is about Geometric Transformations in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Supported coordinate systems / - Floating-point coordinate system for 2D and 3D / - Integer coordinate system (2D) as a special case / - Representation of vectors / - 2D vector by one-dimensional array with 2 elements
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'Geometric Transformations in OpenGL'?
Page 44	OpenGL Transformation Pipeline

Source cue	- Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline
Professor-style explanation	The slide 'OpenGL Transformation Pipeline' describes the OpenGL side of the rendering pipeline. OpenGL represents rendering through objects such as contexts, buffers, vertex arrays, shaders, textures, samplers, and framebuffers. At draw time, the currently bound objects and state determine what data reaches the GPU and how the pipeline processes it. On the slide, the concrete items are: - Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline. The concrete relation is between API state, GPU resource, shader input, and final rendering result.
Technical commentary	This slide is about OpenGL Transformation Pipeline. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. Concrete items shown: - Model-View matrix / - Transformation matrix for model space related transformations / - Additionally contains the transformation of the scene / into the camera coordinate system (view transformation, not projection!) / - Transformation pipeline
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'OpenGL Transformation Pipeline'?

Page 45	Example – Translation and Scaling
Source cue	- Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized
Professor-style explanation	The slide 'Example – Translation and Scaling' introduces a concrete graphics object, operation, or relation. The named terms describe input data, a processing step, and an output used elsewhere in the rendering workflow. On the slide, the concrete items are: - Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized. The object-level relation is therefore input, operation, output, and the later graphics stage that consumes the output.
Technical commentary	This slide is about Example – Translation and Scaling. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Drawing cylinder objects / - drawCylinder draws a cylinder centered at the origin, / with respect to the y-axis, with a given radius and height / - By translation this "unit cylinder" can be positioned / - By scaling this "unit cylinder" can be resized
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Example – Translation and Scaling' into a causal sentence instead of repeating the slide title?
Page 46	Matrix Specification
Source cue	- Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] =;
Professor-style explanation	The slide 'Matrix Specification' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] = . The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about Matrix Specification. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Matrices are often specified in form of a C array / void draw(...) { / ... / GLfloat M[4][4]; / M[0][0] =;
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Matrix Specification'?
Page 47	Example – Portico Modeling 1/5
Source cue	- Columned hall consists of floor space, H·0.1 / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is ■,
Professor-style explanation	The slide 'Example – Portico Modeling 1/5' introduces a concrete graphics object, operation, or relation. The named terms describe input data, a processing step, and an output used elsewhere in the rendering workflow. On the slide, the concrete items are: - Columned hall consists of floor space, H·0.1 / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is ■,. The object-level relation is therefore input, operation, output, and the later graphics stage that consumes the output.
Technical commentary	This slide is about Example – Portico Modeling 1/5. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Columned hall consists of floor space, H·0.1 / top surface and columns / - Columns are evenly distributed / along the edge of the round floor surface H / - Total height of a column is ■,
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example – Portico Modeling 1/5' into a causal sentence instead of repeating the slide title?
Page 48	Example – Portico Modeling 2/5

Source cue	- Drawing a cylinder closed at top and bottom / - Each column should "stand" in the y-axis / - Each column should have a lid surface at the top and bottom / - Disk primitive is embedded in the xy-plane / - Cylinder primitive consists only of lateral surface
Professor-style explanation	The slide 'Example – Portico Modeling 2/5' introduces a concrete graphics object, operation, or relation. The named terms describe input data, a processing step, and an output used elsewhere in the rendering workflow. On the slide, the concrete items are: - Drawing a cylinder closed at top and bottom / - Each column should "stand" in the y-axis / - Each column should have a lid surface at the top and bottom / - Disk primitive is embedded in the xy-plane / - Cylinder primitive consists only of lateral surface. The object-level relation is therefore input, operation, output, and the later graphics stage that consumes the output.
Technical commentary	This slide is about Example – Portico Modeling 2/5. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Drawing a cylinder closed at top and bottom / - Each column should "stand" in the y-axis / - Each column should have a lid surface at the top and bottom / - Disk primitive is embedded in the xy-plane / - Cylinder primitive consists only of lateral surface
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example – Portico Modeling 2/5' into a causal sentence instead of repeating the slide title?
Page 49	Example – Portico Modeling 3/5
Source cue	- Drawing a single column / - Composition of 5 cylinder objects / - Rotation into y-axis to match orientation / - Positioning of the parts by translation along the y-axis / - Incremental translation around the partial column height
Professor-style explanation	The slide 'Example – Portico Modeling 3/5' introduces a concrete graphics object, operation, or relation. The named terms describe input data, a processing step, and an output used elsewhere in the rendering workflow. On the slide, the concrete items are: - Drawing a single column / - Composition of 5 cylinder objects / - Rotation into y-axis to match orientation / - Positioning of the parts by translation along the y-axis / - Incremental translation around the partial column height. The object-level relation is therefore input, operation, output, and the later graphics stage that consumes the output.
Technical commentary	This slide is about Example – Portico Modeling 3/5. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Drawing a single column / - Composition of 5 cylinder objects / - Rotation into y-axis to match orientation / - Positioning of the parts by translation along the y-axis / - Incremental translation around the partial column height
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example – Portico Modeling 3/5' into a causal sentence instead of repeating the slide title?

Page 50	Example – Portico Modeling 4/5
Source cue	- Drawing the group of columns / - Columns are evenly distributed on a circle with radius ■ / - Number ■, radius ■ of the columns as well as floor radius ■ adjustable
Professor-style explanation	The slide 'Example – Portico Modeling 4/5' introduces a concrete graphics object, operation, or relation. The named terms describe input data, a processing step, and an output used elsewhere in the rendering workflow. On the slide, the concrete items are: - Drawing the group of columns / - Columns are evenly distributed on a circle with radius ■ / - Number ■, radius ■ of the columns as well as floor radius ■ adjustable. The object-level relation is therefore input, operation, output, and the later graphics stage that consumes the output.
Technical commentary	This slide is about Example – Portico Modeling 4/5. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Drawing the group of columns / - Columns are evenly distributed on a circle with radius ■ / - Number ■, radius ■ of the columns as well as floor radius ■ adjustable
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example – Portico Modeling 4/5' into a causal sentence instead of repeating the slide title?
Page 51	Example – Portico Modeling 5/5
Source cue	- Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof
Professor-style explanation	The slide 'Example – Portico Modeling 5/5' introduces a concrete graphics object, operation, or relation. The named terms describe input data, a processing step, and an output used elsewhere in the rendering workflow. On the slide, the concrete items are: - Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof. The object-level relation is therefore input, operation, output, and the later graphics stage that consumes the output.
Technical commentary	This slide is about Example – Portico Modeling 5/5. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Drawing the hall / - Use cylinder primitive as floor area / - Use cone primitive as a roof
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Example – Portico Modeling 5/5' into a causal sentence instead of repeating the slide title?
Page 52	by applying affine transformations to their vertices
Source cue	- 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication

Professor-style explanation	The slide 'by applying affine transformations to their vertices' explains how geometric objects change coordinate systems. A point, vector, normal, or local coordinate frame is multiplied by a transformation matrix or affected by an affine operation. Translations move positions, rotations change orientation, scaling changes size, and composed matrices combine several such effects. On the slide, the concrete items are: - 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication. The object-level relation is input coordinate space, transformation object, and output coordinate space.
Technical commentary	This slide is about by applying affine transformations to their vertices. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - 3D models can be positioned, resized and oriented in the 3D world / by applying affine transformations to their vertices / - An affine transformation is represented by a 4x4 matrix / - Affine transformations can be more efficiently applied / when concatenated to a single matrix through matrix multiplication
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'by applying affine transformations to their vertices'?
Page 53	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' introduces a concrete graphics object, operation, or relation. The named terms describe input data, a processing step, and an output used elsewhere in the rendering workflow. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. The object-level relation is therefore input, operation, output, and the later graphics stage that consumes the output.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Literature and other sources used in this chapter' into a causal sentence instead of repeating the slide title?
Page 54	(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)

Source cue	- Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer
Professor-style explanation	The slide '(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)' introduces a concrete graphics object, operation, or relation. The named terms describe input data, a processing step, and an output used elsewhere in the rendering workflow. On the slide, the concrete items are: - Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer. The object-level relation is therefore input, operation, output, and the later graphics stage that consumes the output.
Technical commentary	This slide is about (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Text Books / - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice / (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11) / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn '(3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)' into a causal sentence instead of repeating the slide title?

03.1 Mathematical Foundations

Source location: Section 3.1

Commentary

The lecture uses vectors and matrices to express geometric operations. A vector can represent a point position, a direction, an offset, or an attribute depending on context. A matrix expresses a linear map or, with homogeneous coordinates, an affine transformation.

The central exam habit is to name the coordinate space. A point in model coordinates and a light direction in view coordinates cannot be mixed safely. Many visual bugs are coordinate-space bugs.

Mental model

A numeric vector is incomplete until you know what coordinate system it belongs to.

Check yourself

Why is a normal vector not transformed exactly like a point under all transformations?

03.2 Affine Transformations

Source location: Section 3.2

Commentary

Affine transformations include translation, rotation, scaling, shearing, and combinations of these. Homogeneous coordinates allow translation to be represented in matrix form together with other transformations.

Translation moves positions but not pure directions. Rotation changes orientation while preserving lengths if it is a proper rotation. Scaling can change lengths and can also affect normals. These distinctions matter later in lighting.

Mental model

Use 4D homogeneous coordinates so that a single matrix pipeline can move 3D points through the scene.

Check yourself

What does the homogeneous w component let a transformation matrix express?

03.3 Composition of Transformations

Source location: Section 3.3

Commentary

Composition means applying several transformations in sequence. The order matters because matrix multiplication is generally not commutative. Rotating an object around its own origin and then translating it is different from translating it and then rotating around the world origin.

When you read a formula such as $\text{projection} * \text{view} * \text{model} * \text{position}$, read it from right to left for the point: first model, then view, then projection. The resulting matrix chain is compact, but the conceptual steps remain separate.

Mental model

Matrix chains are compressed stories of movement through coordinate spaces.

Check yourself

Why can swapping model and view matrices destroy the intended camera/object relation?

03.4 Coordinate System Change

Source location: Section 3.4

Commentary

A transformation can be read actively as moving an object, or passively as changing the coordinate frame used to describe it. Both interpretations are valid, but mixing them carelessly causes sign and order errors.

The view matrix is a classic example: conceptually you position a camera, but the pipeline usually transforms the world by the inverse of the camera transform so that the camera becomes the origin of view space.

Mental model

Moving the camera one way is equivalent to moving the world the opposite way for rendering.

Check yourself

Why is the view transform often related to the inverse camera transform?

03.5 Transformations in OpenGL

Source location: Section 3.5

Commentary

Modern OpenGL does not automatically manage the old matrix stacks. You usually compute model, view, and projection matrices in application code and pass them to shaders as uniforms.

The vertex shader is where the final clip-space position is normally computed. That means a wrong matrix uniform, wrong multiplication order, or wrong convention can make geometry vanish even though buffers and draw calls are correct.

Mental model

The shader is the place where abstract transformation math becomes GPU execution.

Check yourself

Which matrix would you inspect first if an object follows the camera instead of staying in the world?

End-of-lecture summary

If you remember only one thing from Lecture 03, remember this: This lecture explains how geometry moves between coordinate systems. Almost every later topic assumes that you can track where a point or vector currently lives.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.